

# The Complete Guide to Building Skills for Claude



# Claude 技能构建完 全指南



# Contents

|                              |    |
|------------------------------|----|
| Introduction                 | 3  |
| Fundamentals                 | 4  |
| Planning and design          | 7  |
| Testing and iteration        | 14 |
| Distribution and sharing     | 18 |
| Patterns and troubleshooting | 21 |
| Resources and references     | 28 |

# 目录

|                |    |
|----------------|----|
| 引言             | 3  |
| 基础             | 4  |
| 规划与设计          | 7  |
| 测试与迭代          | 14 |
| 分发与共享          | 18 |
| 模式与故障排除        | 21 |
| 资源与 references | 28 |

# Introduction

A **skill** is a set of instructions - packaged as a simple folder - that teaches Claude how to handle specific tasks or workflows. Skills are one of the most powerful ways to customize Claude for your specific needs. Instead of re-explaining your preferences, processes, and domain expertise in every conversation, skills let you teach Claude once and benefit every time.

Skills are powerful when you have repeatable workflows: generating frontend designs from specs, conducting research with consistent methodology, creating documents that follow your team's style guide, or orchestrating multi-step processes. They work well with Claude's built-in capabilities like code execution and document creation. For those building MCP integrations, skills add another powerful layer helping turn raw tool access into reliable, optimized workflows.

This guide covers everything you need to know to build effective skills - from planning and structure to testing and distribution. Whether you're building a skill for yourself, your team, or for the community, you'll find practical patterns and real-world examples throughout.

## What you'll learn:

- Technical requirements and best practices for skill structure
- Patterns for standalone skills and MCP-enhanced workflows
- Patterns we've seen work well across different use cases
- How to test, iterate, and distribute your skills

## Who this is for:

- Developers who want Claude to follow specific workflows consistently
- Power users who want Claude to follow specific workflows
- Teams looking to standardize how Claude works across their organization

## Two Paths Through This Guide

Building standalone skills? Focus on Fundamentals, Planning and Design, and category 1-2. Enhancing an MCP integration? The "Skills + MCP" section and category 3 are for you. Both paths share the same technical requirements, but you choose what's relevant to your use case.

**What you'll get out of this guide:** By the end, you'll be able to build a functional skill in a single sitting. Expect about 15-30 minutes to build and test your first working skill using the skill-creator.

Let's get started.

# 引言

技能是一组指令——打包成一个简单的文件夹——用于教导Claude如何处理特定的任务或工作流。技能是为满足您的特定需求而定制Claude的最强大方式之一。无需在每次对话中重新解释您的偏好、流程和领域专业知识，技能让您一次性教导Claude，并在每次使用时受益。

当您拥有可重复的工作流时，技能会非常强大：根据规格生成前端设计、使用一致的方法论进行研究、创建遵循团队风格指南的文档，或编排多步骤流程。它们与Claude的内置Capabilities（如代码执行和文档创建）配合良好。对于那些构建MCP集成的人来说，技能增添了另一个强大的层面，帮助将原始工具访问转化为可靠、优化的工作流。

本指南涵盖了构建有效技能所需了解的一切——从规划与设计到测试与分发。无论你是为自己、团队还是社区构建技能，都能在其中找到实用的模式和真实示例。

## 您将学到：

- 技能结构的技术要求和最佳实践
- 独立技能和MCP增强工作流的模式
- 我们在不同用例中验证有效的模式
- 如何测试、迭代并分发你的技能

## 适用对象：

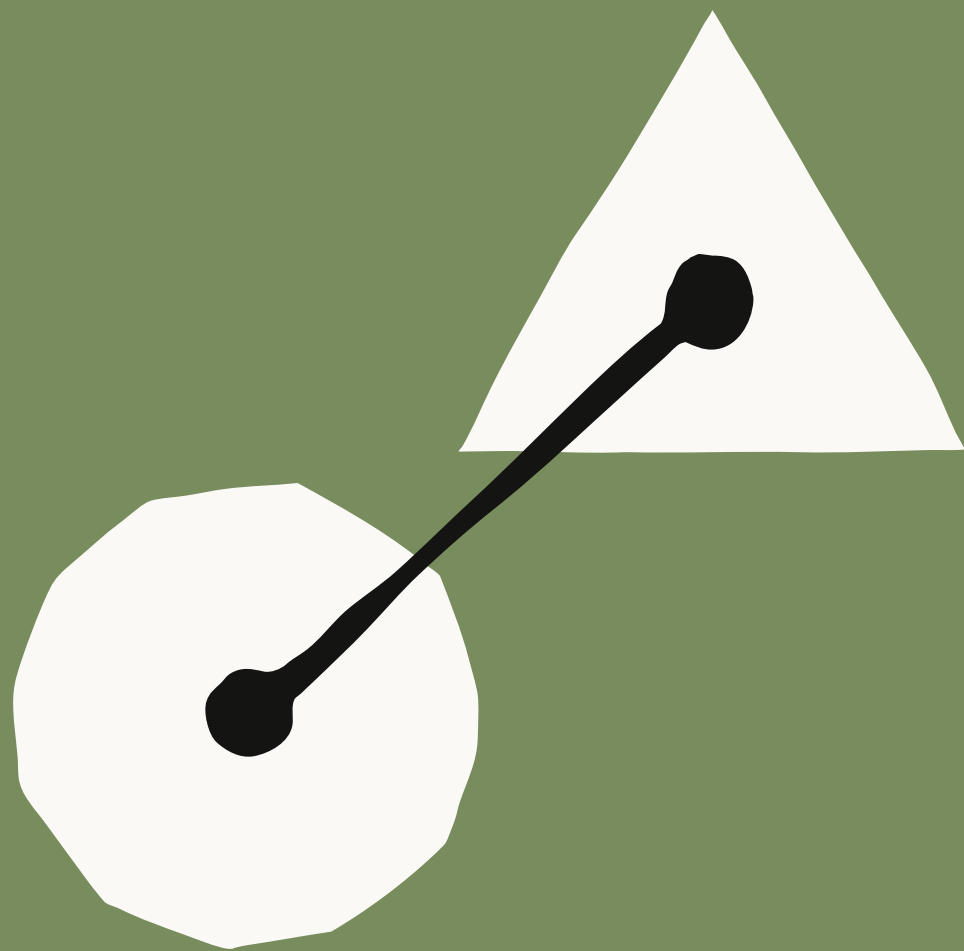
- 希望Claude能持续遵循特定工作流的开发者
- 希望Claude遵循特定工作流的高级用户
- 希望在其组织内标准化Claude工作方式的团队

## 本指南的两条路径

构建独立技能？请重点关注基础、规划与设计以及类别1-2。增强MCP集成？"技能+ MCP"部分和类别3适合你。两条路径共享相同的技术要求，但你可以选择与自身用例相关的部分。

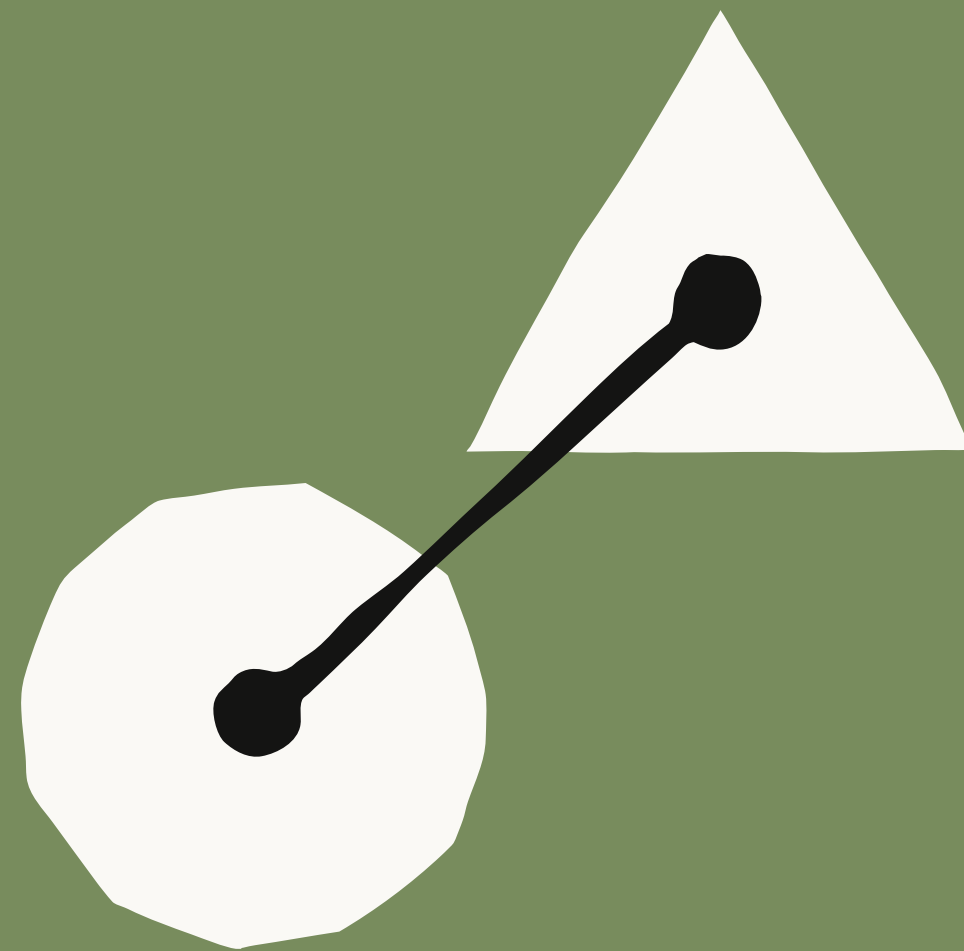
你将从本指南中获得什么：到最后，你将能够在一次会话中构建一个功能性的技能。预计使用skill-creator构建和测试你的第一个可用技能大约需要15-30分钟。

让我们开始吧。



Chapter 1

# Fundamentals



第一章

# 基础

# Fundamentals

## What is a skill?

A skill is a folder containing:

- **SKILL.md** (required): Instructions in Markdown with YAML frontmatter
- **scripts/** (optional): Executable code (Python, Bash, etc.)
- **references/** (optional): Documentation loaded as needed
- **assets/** (optional): Templates, fonts, icons used in output

## Core design principles

### Progressive Disclosure

Skills use a three-level system:

- **First level (YAML frontmatter):** Always loaded in Claude's system prompt. Provides just enough information for Claude to know when each skill should be used without loading all of it into context.
- **Second level (SKILL.md body):** Loaded when Claude thinks the skill is relevant to the current task. Contains the full instructions and guidance.
- **Third level (Linked files):** Additional files bundled within the skill directory that Claude can choose to navigate and discover only as needed.

This progressive disclosure minimizes token usage while maintaining specialized expertise.

## Composability

Claude can load multiple skills simultaneously. Your skill should work well alongside others, not assume it's the only capability available.

### Portability

Skills work identically across Claude.ai, Claude Code, and API. Create a skill once and it works across all surfaces without modification, provided the environment supports any dependencies the skill requires.

## For MCP Builders: Skills + Connectors

💡 *Building standalone skills without MCP? Skip to Planning and Design - you can always return here later.*

If you already have a **working MCP server**, you've done the hard part. Skills are the knowledge layer on top - capturing the workflows and best practices you already know, so Claude can apply them consistently.

## The kitchen analogy

**MCP provides the professional kitchen:** access to tools, ingredients, and equipment.

**Skills provide the recipes:** step-by-step instructions on how to create something valuable.

# 基础

## 什么是技能？

技能是一个包含以下内容的文件夹：

- SKILL.md（必需）：包含 YAML 前置元数据的 Markdown 格式说明
- scripts/（可选）：可执行代码（Python、Bash 等）
- references/（可选）：按需加载的文档
- assets/（可选）：输出中使用的模板、字体、图标

## 核心设计原则

### 渐进式披露

技能采用三级系统：

- 第一级（YAML 前置元数据）：始终加载到 Claude 的系统提示词中。提供足够的信息让 Claude 知道何时应使用每个技能，而不需要将所有内容加载到上下文中。
- 第二层（SKILL.md正文）：当Claude认为该技能与当前任务相关时加载。包含完整的指令和指导信息。

技能在Claude.ai、Claude Code和API中工作方式完全相同。只需创建一次技能，无需修改即可在所有平台上使用，前提是环境支持该技能所需的任何依赖项。

Claude可以根据需要选择性地浏览和发现这些文件。

这种渐进式披露能在保持专业专长的同时，最小化令牌使用量。

## 可组合性

Claude 可以同时加载多个技能。您的技能应与其他技能良好协作，不应假设它是唯一可用的能力。

### 可移植性

Skills work identically across Claude.ai, Claude Code, and API. Create a skill once and it works across all surfaces without modification, provided the environment supports any dependencies the skill requires.

## 致 MCP 构建者：技能 + 连接器

💡 正在构建不依赖 MCP 的独立技能？请跳转至规划与设计部分——您可以随时返回此处。

如果您已经拥有一个可运行的 MCP 服务器，那么您已经完成了最困难的部分。技能是位于其上的知识层——用于捕获您已知的工作流和最佳实践，以便 Claude 能够一致地应用它们。

## 厨房类比

MCP 提供了专业厨房：可以访问工具、食材和设备。

技能提供食谱：关于如何创造有价值事物的分步说明。

Together, they enable users to accomplish complex tasks without needing to figure out every step themselves.

How they work together:

| MCP (Connectivity)                                            | Skills (Knowledge)                                 |
|---------------------------------------------------------------|----------------------------------------------------|
| Connects Claude to your service (Notion, Asana, Linear, etc.) | Teaches Claude how to use your service effectively |
| Provides real-time data access and tool invocation            | Captures workflows and best practices              |
| What Claude can do                                            | How Claude should do it                            |

## Why this matters for your MCP users

### Without skills:

- Users connect your MCP but don't know what to do next
- Support tickets asking "how do I do X with your integration"
- Each conversation starts from scratch
- Inconsistent results because users prompt differently each time
- Users blame your connector when the real issue is workflow guidance

### With skills:

- Pre-built workflows activate automatically when needed
- Consistent, reliable tool usage
- Best practices embedded in every interaction
- Lower learning curve for your integration

它们共同让用户无需自行摸索每一步，就能完成复杂的任务。

它们如何协同工作：

| MCP (连接性)                                | 技能 (知识)              |
|------------------------------------------|----------------------|
| 将 Claude 连接到您的服务 (Notion、Asana、Linear 等) | 教导 Claude 如何有效使用您的服务 |
| 提供实时数据访问和工具调用                            | 捕获工作流和最佳实践           |
| Claude 能做什么                              | Claude 应该如何做         |

## 这对您的 MCP用户 为何重要

### 没有技能时：

- 用户连接了您的 MCP，但不知道下一步该做什么
- 询问“如何通过您的集成实现 X”的支持工单
- 每次对话都从头开始
- 结果不一致，因为用户每次的提示方式不同
- 用户责怪您的连接器，而真正的问题在于工作流指导

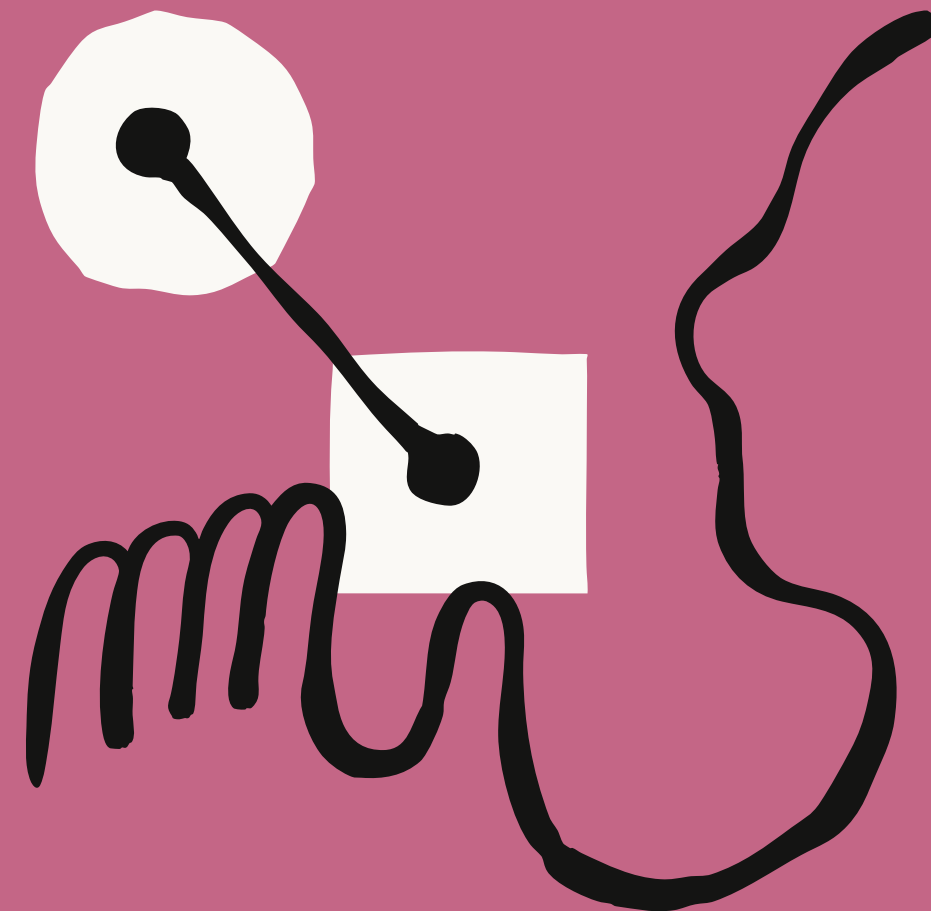
### 具备技能：

- 预构建工作流会在需要时自动激活
- 一致、可靠的工具使用
- 最佳实践嵌入于每一次交互中
- 降低您的集成学习曲线



Chapter 2

# Planning and design



第二章

# 规划与设计

# Planning and design

## Start with use cases

Before writing any code, identify 2-3 concrete use cases your skill should enable.

### Good use case definition:

Use Case: Project Sprint Planning

Trigger: User says "help me plan this sprint" or "create sprint tasks"

Steps:

1. Fetch current project status from Linear (via MCP)
2. Analyze team velocity and capacity
3. Suggest task prioritization
4. Create tasks in Linear with proper labels and estimates

Result: Fully planned sprint with tasks created

### Ask yourself:

- What does a user want to accomplish?
- What multi-step workflows does this require?
- Which tools are needed (built-in or MCP?)
- What domain knowledge or best practices should be embedded?

## Common skill use case categories

At Anthropic, we’ve observed three common use cases:

### Category 1: Document & Asset Creation

Used for: Creating consistent, high-quality output including documents, presentations, apps, designs, code, etc.

Real example: frontend-design skill (also see skills for docx, pptx, xlsx, and ppt)

"Create distinctive, production-grade frontend interfaces with high design quality. Use when building web components, pages, artifacts, posters, or applications."

#### Key techniques:

- Embedded style guides and brand standards
- Template structures for consistent output
- Quality checklists before finalizing
- No external tools required - uses Claude's built-in capabilities

# 规划与设计

## 从用例开始

在编写任何代码之前，先确定你的技能应能实现的2-3个具体用例。

### 良好的用例定义：

Use Case: Project Sprint Planning

Trigger: User says "help me plan this sprint" or "create sprint tasks"

Steps:

1. Fetch current project status from Linear (via MCP)
2. Analyze team velocity and capacity
3. Suggest task prioritization
4. Create tasks in Linear with proper labels and estimates

Result: Fully planned sprint with tasks created

### 问问自己：

- 用户想要实现什么目标？
- 这需要哪些多步骤工作流？
- 需要哪些工具（内置工具还是MCP? ）
- 应嵌入哪些领域知识或最佳实践？

## 常见技能用例类别

在Anthropic，我们观察到三种常见用例：

### 类别 1： 文档与资源创建

用途：创建一致、高质量的输出，包括文档、演示文稿、应用程序、设计、代码等。

实际示例：前端设计技能（另请参阅 DOCX、PPTX、XLSX 和 PPT 相关技能）

"创建独特、生产级的高质量设计前端界面。适用于构建Web组件、页面、工件、海报或应用程序。"

#### 关键技术：

- Embedded style guides and brand standards
- 用于一致输出的模板结构
- 最终确定前的质量检查清单
- 无需外部工具 - 使用 Claude 的内置 Capabilities

## Category 2: Workflow Automation

Used for: Multi-step processes that benefit from consistent methodology, including coordination across multiple MCP servers.

*Real example:* **skill-creator skill**

"Interactive guide for creating new skills. Walks the user through use case definition, frontmatter generation, instruction writing, and validation."

### Key techniques:

- Step-by-step workflow with validation gates
- Templates for common structures
- Built-in review and improvement suggestions
- Iterative refinement loops

## Category 3: MCP Enhancement

Used for: Workflow guidance to enhance the tool access an MCP server provides.

*Real example:* **sentry-code-review skill (from Sentry)**

"Automatically analyzes and fixes detected bugs in GitHub Pull Requests using Sentry's error monitoring data via their MCP server."

### Key techniques:

- Coordinates multiple MCP calls in sequence
- Embeds domain expertise
- Provides context users would otherwise need to specify
- Error handling for common MCP issues

## Define success criteria

### How will you know your skill is working?

These are aspirational targets - rough benchmarks rather than precise thresholds. Aim for rigor but accept that there will be an element of vibes-based assessment. We are actively developing more robust measurement guidance and tooling.

### Quantitative metrics:

- Skill triggers on 90% of relevant queries
  - *How to measure:* Run 10-20 test queries that should trigger your skill. Track how many times it loads automatically vs. requires explicit invocation.
- Completes workflow in X tool calls
  - *How to measure:* Compare the same task with and without the skill enabled. Count tool calls and total tokens consumed.
- 0 failed API calls per workflow
  - *How to measure:* Monitor MCP server logs during test runs. Track retry rates and error codes.

### Qualitative metrics:

- Users don't need to prompt Claude about next steps
  - *How to assess:* During testing, note how often you need to redirect or clarify. Ask beta users for feedback.
- Workflows complete without user correction
  - *How to assess:* Run the same request 3-5 times. Compare outputs for structural consistency and quality.
- Consistent results across sessions
  - *How to assess:* Can a new user accomplish the task on first try with minimal guidance?

## 类别 2：工作流自动化

用途：适用于受益于一致方法论的多步骤流程，包括跨多个MCP服务器的协调工作。

真实示例：skill-creator 技能

"创建新技能的交互式指南。引导用户完成用例定义、前置元数据生成、指令编写和验证。"

### 关键技术：

- 包含验证关卡的逐步工作流
- 用于常见结构的模板
- 内置审查与改进建议
- 迭代优化循环

## 类别 3：MCP 增强

用途：提供工作流指导，以增强 MCP 服务器所提供的工具访问能力。

实际示例：sentry-code-review 技能（来自 Sentry）

"通过其 MCP 服务器，利用 Sentry 的错误监控数据，自动分析并修复 GitHub 拉取请求中检测到的错误。"

### 关键技术：

- 按顺序协调多个MCP调用
- 嵌入领域专业知识
- 提供用户通常需要另行指定的上下文
- 针对常见MCP问题的错误处理

## 定义成功标准

### 如何确认您的技能正在有效运作？

这些是理想目标——粗略的基准而非精确阈值。力求严谨，但需接受评估中会存在一定基于直觉的成分。我们正在积极开发更稳健的测量指南与工具。

### 量化指标：

- 技能在90%的相关查询中触发
  - 测量方法：运行10-20条应触发您技能的测试查询。追踪其自动加载的次数与需要显式调用的次数。
- 在X次工具调用中完成工作流
  - 测量方法：对比启用与未启用技能时执行同一任务的情况。统计工具调用次数及消耗的令牌总数。
- 0 每个工作流失败的API调用
  - 衡量方法：在测试运行期间监控MCP服务器日志。跟踪重试率和错误代码。

### 定性指标：

- 用户无需提示Claude下一步操作
  - 评估方法：在测试期间，记录你需要重新引导或澄清的频率。向测试用户征求反馈。
- 工作流无需用户修正即可完成
  - 评估方法：运行同一请求 3-5 次。比较输出的结构一致性和质量。
- 跨会话结果一致
  - 评估方法：新用户能否在首次尝试时，以最少的指导完成该任务？

## Technical requirements

### File structure

```
your-skill-name/
├── SKILL.md           # Required - main skill file
├── scripts/           # Optional - executable code
│   ├── process_data.py # Example
│   └── validate.sh # Example
├── references/        # Optional - documentation
│   ├── api-guide.md # Example
│   └── examples/ # Example
└── assets/            # Optional - templates, etc.
    └── report-template.md # Example
```

### Critical rules

#### SKILL.md naming:

- Must be exactly **SKILL.md** (case-sensitive)
- No variations accepted (SKILL.MD, skill.md, etc.)

#### Skill folder naming:

- Use kebab-case: **notion-project-setup** ✅
- No spaces: **Notion Project Setup** ❌
- No underscores: **notion\_project\_setup** ❌
- No capitals: **NotionProjectSetup** ❌

#### No README.md:

- Don't include README.md inside your skill folder
- All documentation goes in SKILL.md or references/
- Note: when distributing via GitHub, you'll still want a repo-level README for human users — see Distribution and Sharing.

## YAML frontmatter: The most important part

The YAML frontmatter is how Claude decides whether to load your skill. Get this right.

### Minimal required format

```
---
name: your-skill-name
description: What it does. Use when user asks to [specific
phrases].
---
```

That's all you need to start.

### Field requirements

#### name (required):

- kebab-case only
- No spaces or capitals
- Should match folder name

#### description (required):

- **MUST include BOTH:**
  - What the skill does
  - When to use it (trigger conditions)
- Under 1024 characters
- No XML tags (< or >)
- Include specific tasks users might say
- Mention file types if relevant

## 技术要求

### 文件结构

```
your-skill-name/
├── SKILL.md           # Required - main skill file
├── scripts/           # Optional - executable code
│   ├── process_data.py # Example
│   └── validate.sh # Example
├── references/        # Optional - documentation
│   ├── api-guide.md # Example
│   └── examples/ # Example
└── assets/            # Optional - templates, etc.
    └── report-template.md # Example
```

### 关键规则

#### SKILL.md命名:

- 必须完全为**SKILL.md** (区分大小写)
- 不接受任何变体 (SKILL.md、技能.md等)

#### Skill 文件夹命名:

- 使用短横线命名法: **notion-project-setup** ✅
- 无空格: **Notion Project Setup** ❌
- 无下划线: **notion\_project\_setup** ❌
- 无大写字母: **NotionProjectSetup** ❌

#### 无 README.md:

- 不要在您的 Skill 文件夹中包含 README.md
- 所有文档都应放在 SKILL.md 或 references/ 目录中
- 注意: 通过 GitHub 分发时, 您仍需要一个仓库级 README 供人类用户使用——请参阅分发与共享。

## YAML 前置元数据: 最重要的部分

YAML 前置元数据是 Claude 决定是否加载您技能的依据。请务必正确填写。

### 必需的最小格式

```
---
name: your-skill-name
description: What it does. Use when user asks to [specific
phrases].
---
```

这就是你开始所需的一切。

### 字段要求

#### 姓名 (必需):

- 仅限短横线命名法
- 无空格或大写字母
- 应与文件夹名称匹配

#### description (必需):

- 必须包含以下两者:
  - 该技能的功能
  - 何时使用 (触发条件)
- 少于1024个字符
- 无XML标签 (< 或 >)
- 包含用户可能提及的具体任务
- 提及相关的文件类型

**license** (optional):

- Use if making skill open source
- Common: MIT, Apache-2.0

**compatibility** (optional)

- 1-500 characters
- Indicates environment requirements: e.g. intended product, required system packages, network access needs, etc.

**metadata** (optional):

- Any custom key-value pairs
- Suggested: author, version, mcp-server
- *Example:*

```
```yaml
metadata:
  author: ProjectHub
  version: 1.0.0 mcp-server: projecthub
```
```

### Security restrictions

**Forbidden in frontmatter:**

- XML angle brackets (< >)
- Skills with "claude" or "anthropic" in name (reserved)

**Why:** Frontmatter appears in Claude's system prompt. Malicious content could inject instructions.

## Writing effective skills

### The description field

According to Anthropic's **engineering blog**: "This metadata...provides just enough information for Claude to know when each skill should be used without loading all of it into context." This is the first level of progressive disclosure.

**Structure:**

[What it does] + [When to use it] + [Key capabilities]

**Examples of good descriptions:**

```
# Good - specific and actionable
description: Analyzes Figma design files and generates
developer handoff documentation. Use when user uploads .fig
files, asks for "design specs", "component documentation", or
"design-to-code handoff".
```

```
# Good - includes trigger phrases
description: Manages Linear project workflows including sprint
planning, task creation, and status tracking. Use when user
mentions "sprint", "Linear tasks", "project planning", or asks
to "create tickets".
```

```
# Good - clear value proposition
description: End-to-end customer onboarding workflow for
PayFlow. Handles account creation, payment setup, and
subscription management. Use when user says "onboard new
customer", "set up subscription", or "create PayFlow account".
```

许可证（可选）：

- 若将技能开源时使用
- 常见：MIT、Apache-2.0

兼容性（可选）

- 1-500字符
- 指明环境要求：例如目标产品、所需系统包、网络访问需求等。

元数据（可选）：

- 任何自定义键值对
- 建议：作者、版本、mcp-server
- 示例：

```
```yaml
metadata:
  author: ProjectHub
  version: 1.0.0 mcp-server: projecthub
```
```

### 安全限制

前置元数据中禁止包含：

- XML 尖括号 (< >)
- 名称中包含“Claude”或“Anthropic”的技能（保留）

原因：前置元数据出现在 Claude 的系统提示词中。恶意内容可能会注入指令。

## 编写有效的技能

### 描述字段

根据 Anthropic 的工程博客所述：“此元数据……为 Claude 提供了足够的信息，使其知道何时应使用每种技能，而不必将所有内容加载到上下文中。”这是渐进式披露的第一层。

结构：

[What it does] + [When to use it] + [Key capabilities]

**Examples of good descriptions:**

```
# Good - specific and actionable
良好描述的示例：
```

```
# Good - includes trigger phrases
description: Manages Linear project workflows including sprint
planning, task creation, and status tracking. Use when user
mentions "sprint", "Linear tasks", "project planning", or asks
to "create tickets".
```

```
# Good - clear value proposition
description: End-to-end customer onboarding workflow for
PayFlow. Handles account creation, payment setup, and
subscription management. Use when user says "onboard newcustomer",
"set up subscription", or "create PayFlow account".
```

Examples of bad descriptions:

```
# Too vague
description: Helps with projects.

# Missing triggers
description: Creates sophisticated multi-page documentation systems.

# Too technical, no user triggers
description: Implements the Project entity model with hierarchical relationships.
```

Writing the main instructions

After the frontmatter, write the actual instructions in Markdown.

Recommended structure:

Adapt this template for your skill. Replace bracketed sections with your specific content.

```
---
name: your-skill
description: [...]
---

# Your Skill Name

## Instructions

### Step 1: [First Major Step]
Clear explanation of what happens.
```

```
Example:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Expected output: [describe what success looks like]
```

(Add more steps as needed)

Examples

Example 1: [common scenario]

User says: "Set up a new marketing campaign"

Actions:

- 1. Fetch existing campaigns via MCP
- 2. Create new campaign with provided parameters

Result: Campaign created with confirmation link

(Add more examples as needed)

Troubleshooting

Error: [Common error message]

Cause: [Why it happens]

Solution: [How to fix]

(Add more error cases as needed)

不良描述示例:

```
# Too vague
description: Helps with projects.

# Missing triggers
description: Creates sophisticated multi-page documentation systems.

# Too technical, no user triggers
description: Implements the Project entity model with hierarchical relationships.
```

编写主要指令

在前置元数据之后，使用 Markdown 编写实际的指令。

推荐结构:

请根据您的技能调整此模板。将方括号内的部分替换为您具体的内容。

```
---
name: your-skill
description: [...]
---

# Your Skill Name

## Instructions

### Step 1: [First Major Step]
Clear explanation of what happens.
```

```
Example:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Expected output: [describe what success looks like]
```

(根据需要添加更多步骤)

示例

示例 1: [常见场景]

用户说：“设置一个新的营销活动”

操作:

- 1. Fetch existing campaigns via MCP
- 2. Create new campaign with provided parameters

结果：活动已创建，附带确认链接

(根据需要添加更多示例)

故障排除

错误: [常见错误信息]

原因: [为何发生]

解决方案: [如何修复]

(根据需要添加更多错误案例)

## Best Practices for Instructions

### Be Specific and Actionable

✅ Good:

```
Run `python scripts/validate.py --input {filename}` to check data format.
If validation fails, common issues include:
- Missing required fields (add them to the CSV)
- Invalid date formats (use YYYY-MM-DD)
```

❌ Bad:

```
Validate the data before proceeding.
```

### Include error handling

```
## Common Issues

### MCP Connection Failed
If you see "Connection refused":
1. Verify MCP server is running: Check Settings > Extensions
2. Confirm API key is valid
3. Try reconnecting: Settings > Extensions > [Your Service] > Reconnect
```

## Reference bundled resources clearly

```
Before writing queries, consult `references/api-patterns.md` for:
- Rate limiting guidance
- Pagination patterns
- Error codes and handling
```

### Use progressive disclosure

Keep SKILL.md focused on core instructions. Move detailed documentation to [`references/`](#) and link to it. (See [Core Design Principles](#) for how the three-level system works.)

## 指令的最佳实践

### 具体且可操作

✅ 良好示例:

```
Run `python scripts/validate.py --input {filename}` to check data format.
If validation fails, common issues include:
- Missing required fields (add them to the CSV)
- Invalid date formats (use YYYY-MM-DD)
```

❌ 不良示例:

```
Validate the data before proceeding.
```

### 包含错误处理

```
## Common Issues

### MCP Connection Failed
If you see "Connection refused":
1. Verify MCP server is running: Check Settings > Extensions
2. Confirm API key is valid
3. Try reconnecting: Settings > Extensions > [Your Service] > Reconnect
```

## 明确引用捆绑资源

```
Before writing queries, consult `references/api-patterns.md` for:
- Rate limiting guidance
- Pagination patterns
- Error codes and handling
```

### 使用渐进式披露

保持SKILL.md专注于核心说明。将详细文档移至[`references/`](#)并链接至该处。（参阅核心设计原则了解三级系统的工作原理。）



Chapter 3

# Testing and iteration



第三章

# 测试与迭代

# Testing and iteration

Skills can be tested at varying levels of rigor depending on your needs:

- **Manual testing in Claude.ai** - Run queries directly and observe behavior. Fast iteration, no setup required.
- **Scripted testing in Claude Code** - Automate test cases for repeatable validation across changes.
- **Programmatic testing via skills API** - Build evaluation suites that run systematically against defined test sets.

Choose the approach that matches your quality requirements and the visibility of your skill. A skill used internally by a small team has different testing needs than one deployed to thousands of enterprise users.

**Pro Tip:** Iterate on a single task before expanding

We’ve found that the most effective skill creators iterate on a single challenging task until Claude succeeds, then extract the winning approach into a skill. This leverages Claude’s in-context learning and provides faster signal than broad testing. Once you have a working foundation, expand to multiple test cases for coverage.

## Recommended Testing Approach

Based on early experience, effective skills testing typically covers three areas:

### 1. Triggering tests

**Goal:** Ensure your skill loads at the right times.

**Test cases:**

-  Triggers on obvious tasks
-  Triggers on paraphrased requests
-  Doesn't trigger on unrelated topics

**Example test suite:**

```
Should trigger:  
- "Help me set up a new ProjectHub workspace"  
- "I need to create a project in ProjectHub"  
- "Initialize a ProjectHub project for Q4 planning"
```

```
Should NOT trigger:  
- "What's the weather in San Francisco?"  
- "Help me write Python code"  
- "Create a spreadsheet" (unless ProjectHub skill handles sheets)
```

# 测试与迭代

可根据需求以不同严谨程度测试技能：

- 在 Claude.ai 中进行手动测试 - 直接运行查询并观察行为。快速迭代， 无需设置。
- **在 Claude Code 中进行脚本化测试** - 自动化测试用例，以便在变更中实现可重复的验证。
- **Programmatic testing via skills API** - Build evaluation suites that run systematically against defined test sets.

请根据您的质量要求和技能的可见性来选择相应的方法。一个由小型团队内部使用的技能， 其测试需求与部署给数千名企业用户的技能截然不同。

专业建议：先针对单个任务进行迭代， 再扩展范围

我们发现，最有效的技能创建者会针对一个具有挑战性的任务进行反复迭代，直到Claude成功，然后将获胜的方法提取为一项技能。这利用了Claude的上下文学习能力， 并且比广泛的测试能提供更快的反馈信号。一旦你有了一个可用的基础， 就可以扩展到多个测试用例以实现覆盖。

## 推荐的测试方法

根据早期经验， 有效的技能测试通常涵盖三个领域：

### 1. 触发测试

目标： 确保您的技能在正确时机加载。

测试用例：

-  触发明显任务
-  在改写请求时触发
-  对无关主题不触发

示例测试套件：

```
Should trigger:  
- "Help me set up a new ProjectHub workspace"  
- "I need to create a project in ProjectHub"  
- "Initialize a ProjectHub project for Q4 planning"
```

```
Should NOT trigger:  
- "What's the weather in San Francisco?"  
- "Help me write Python code"  
- "Create a spreadsheet" (unless ProjectHub skill handles sheets)
```

2. Functional tests

**Goal:** Verify the skill produces correct outputs.

**Test cases:**

- Valid outputs generated
- API calls succeed
- Error handling works
- Edge cases covered

**Example:**

Test: Create project with 5 tasks  
Given: Project name "Q4 Planning", 5 task descriptions  
When: Skill executes workflow  
Then:  
- Project created in ProjectHub  
- 5 tasks created with correct properties  
- All tasks linked to project  
- No API errors

3. Performance comparison

**Goal:** Prove the skill improves results vs. baseline.

Use the metrics from Define Success Criteria. Here's what a comparison might look like.

**Baseline comparison:**

Without skill:  
- User provides instructions each time  
- 15 back-and-forth messages  
- 3 failed API calls requiring retry  
- 12,000 tokens consumed

With skill:  
- Automatic workflow execution  
- 2 clarifying questions only  
- 0 failed API calls  
- 6,000 tokens consumed

Using the skill-creator skill

The **skill-creator** skill - available in Claude.ai via plugin directory or download for Claude Code - can help you build and iterate on skills. If you have an MCP server and know your top 2-3 workflows, you can build and test a functional skill in a single sitting - often in 15-30 minutes.

**Creating skills:**

- Generate skills from natural language descriptions
- Produce properly formatted SKILL.md with frontmatter
- Suggest trigger phrases and structure

**Reviewing skills:**

- Flag common issues (vague descriptions, missing triggers, structural problems)
- Identify potential over/under-triggering risks
- Suggest test cases based on the skill's stated purpose

**Iterative improvement:**

- After using your skill and encountering edge cases or failures, bring those examples back to skill-creator
- Example: "Use the issues & solution identified in this chat to improve how the skill handles [specific edge case]"

2. 功能测试

目标：验证技能能产生正确的输出。

测试用例：

- 有效输出已生成
- API调用成功
- 错误处理生效
- 边缘情况已覆盖

示例：

Test: Create project with 5 tasks  
Given: Project name "Q4 Planning", 5 task descriptions  
When: Skill executes workflow  
Then:  
- Project created in ProjectHub  
- 5 tasks created with correct properties  
- All tasks linked to project  
- No API errors

3. 性能比较

目标：证明该技能相比基线能改善结果。

使用“定义成功标准”中的指标。以下是比较可能呈现的样子。

基线比较：

Without skill:  
- User provides instructions each time  
- 15 back-and-forth messages  
- 3 failed API calls requiring retry  
- 12,000 tokens consumed

With skill:  
- Automatic workflow execution  
- 2 clarifying questions only  
- 0 failed API calls  
- 6,000 tokens consumed

使用技能创建器技能

该**skill-creator** 技能——可通过插件目录在Claude.ai获取或为Claude Code下载——能帮助你构建并迭代技能。如果你拥有一个MCP服务器并清楚你最常用的2-3个工作流，你可以在一次会话中构建并测试一个功能性技能——通常只需15-30分钟。

创建技能：

- 根据自然语言描述生成技能
- 生成带有前置元数据的格式正确的SKILL.md
- 建议触发短语和结构

审查技能：

- 标记常见问题（描述模糊、缺少触发、结构问题）
- 识别潜在的过度/欠触发风险
- 根据技能的既定目的建议测试用例

迭代改进：

- 在使用你的技能并遇到边缘情况或失败后，将这些示例带回给 skill-creator
- 示例：“利用本次聊天中发现的问题与解决方案，改进该技能处理[特定边缘情况]的方式”

To use:

"Use the skill-creator skill to help me build a skill for [your use case]"

*Note: skill-creator helps you design and refine skills but does not execute automated test suites or produce quantitative evaluation results.*

Iteration based on feedback

Skills are living documents. Plan to iterate based on:

Undertriggering signals:

- Skill doesn't load when it should
- Users manually enabling it
- Support questions about when to use it

**Solution:** Add more detail and nuance to the description - this may include keywords particularly for technical terms

Overtriggering signals:

- Skill loads for irrelevant queries
- Users disabling it
- Confusion about purpose

**Solution:** Add negative triggers, be more specific

Execution issues:

- Inconsistent results
- API call failures
- User corrections needed

**Solution:** Improve instructions, add error handling

使用方法:

"Use the skill-creator skill to help me build a skill for [your use case]"

注意: skill-creator 帮助您设计和优化技能, 但不执行自动化测试套件或生成定量评估结果。

基于反馈的迭代

技能是动态文档。计划基于以下因素进行迭代:

触发不足信号:

- 技能在应该加载时未加载
- 用户手动启用它
- 关于何时使用它的支持问题

解决方案: 在description中添加更多细节和细微差别——这可能包括针对技术术语的关键词

过度触发信号:

- 技能为无关查询加载
- 用户禁用它
- 对目的感到困惑

解决方案: 添加否定触发词, 更具体

执行问题:

- 结果不一致
- API调用失败
- 需要用户纠正

解决方案: 改进指令, 添加错误处理



# Distribution and sharing



# 分发与共享

# Distribution and sharing

Skills make your MCP integration more complete. As users compare connectors, those with skills offer a faster path to value, giving you an edge over MCP-only alternatives.

## Current distribution model (January 2026)

### How individual users get skills:

- 1. Download the skill folder
- 2. Zip the folder (if needed)
- 3. Upload to Claude.ai via Settings > Capabilities > Skills
- 4. Or place in Claude Code skills directory

### Organization-level skills:

- Admins can deploy skills workspace-wide (shipped December 18, 2025)
- Automatic updates
- Centralized management

## An open standard

We've published **Agent Skills** as an open standard. Like MCP, we believe skills should be portable across tools and platforms - the same skill should work whether you're using Claude or other AI platforms. That said, some skills are designed to take full advantage of a specific platform's capabilities; authors can note this in the skill's **compatibility** field. We've been collaborating with members of the ecosystem on the standard, and we're excited by early adoption.

## Using skills via API

For programmatic use cases - such as building applications, agents, or automated workflows that leverage skills - the API provides direct control over skill management and execution.

### Key capabilities:

- `/v1/skills` endpoint for listing and managing skills
- Add skills to Messages API requests via the `container.skills` parameter
- Version control and management through the Claude Console
- Works with the Claude Agent SDK for building custom agents

### When to use skills via the API vs. Claude.ai:

| Use Case                                        | Best Surface            |
|-------------------------------------------------|-------------------------|
| End users interacting with skills directly      | Claude.ai / Claude Code |
| Manual testing and iteration during development | Claude.ai / Claude Code |
| Individual, ad-hoc workflows                    | Claude.ai / Claude Code |
| Applications using skills programmatically      | API                     |
| Production deployments at scale                 | API                     |
| Automated pipelines and agent systems           | API                     |

# 分发与共享

技能让你的 MCP 集成更加完善。当用户比较连接器时，那些具备技能的连接器能提供更快的价值实现路径，让你相较于仅支持 MCP 的替代方案更具优势。

## 当前分发模型（2026年1月）

### 个人用户如何获取技能：

- 1. 下载 Skill 文件夹
- 2. 压缩文件夹（如果需要）
- 3. 通过设置 > Capabilities > 技能上传到 Claude.ai
- 4. 或放置在 Claude Code 技能目录中

### 组织级技能：

- 管理员可以部署整个工作空间的技能（发布于 2025年12月18日）
- 自动更新
- 集中化管理

## 一个开放标准

我们已将智能体技能发布为开放标准。与MCP一样，我们认为技能应该能够在不同工具和平台间移植——同一个技能应该能够无论您使用的是Claude还是其他AI平台。当然，有些技能旨在充分利用特定平台的Capabilities；作者可以在技能的**compatibility**字段中注明这一点。我们一直在与生态系统成员合作制定该标准，并对早期的采纳感到兴奋。

## 通过API使用技能

对于程序化用例——例如构建应用程序、代理或自动化工作流以利用技能——API提供了对技能管理和执行的直接控制。

### 关键Capabilities：

- `/v1/skills` 用于列出和管理技能的端点
- 通过`container.skills` 参数向 Messages API 请求添加技能
- 通过 Claude 控制台进行版本控制与管理
- 与 Claude Agent SDK 配合使用，用于构建自定义 Agent

### 何时通过 API 使用技能 vs. Claude.ai：

| 使用场景           | 最佳使用界面                  |
|----------------|-------------------------|
| 终端用户直接与技能交互    | Claude.ai / Claude Code |
| 开发过程中的手动测试与迭代  | Claude.ai / Claude Code |
| 个人、临时工作流       | Claude.ai / Claude Code |
| 应用程序以程序化方式使用技能 | API                     |
| 大规模生产部署        | API                     |
| 自动化流水线与智能体系统   | API                     |

**Note:** Skills in the API require the Code Execution Tool beta, which provides the secure environment skills need to run.

For implementation details, see:

- [Skills API Quickstart](#)
- [Create Custom skills](#)
- [Skills in the Agent SDK](#)

## Recommended approach today

Start by hosting your skill on GitHub with a public repo, clear README (for human visitors — this is separate from your skill folder, which should not contain a README.md), and example usage with screenshots. Then add a section to your MCP documentation that links to the skill, explains why using both together is valuable, and provides a quick-start guide.

### 1. Host on GitHub

- Public repo for open-source skills
- Clear README with installation instructions
- Example usage and screenshots

### 2. Document in Your MCP Repo

- Link to skills from MCP documentation
- Explain the value of using both together
- Provide quick-start guide

### 3. Create an Installation Guide

```
## Installing the [Your Service] skill

1. Download the skill:
  - Clone repo: `git clone https://github.com/yourcompany/skills`
  - Or download ZIP from Releases

2. Install in Claude:
  - Open Claude.ai > Settings > skills
  - Click "Upload skill"
```

- Select the skill folder (zipped)

#### 3. Enable the skill:

- Toggle on the [Your Service] skill
- Ensure your MCP server is connected

#### 4. Test:

- Ask Claude: "Set up a new project in [Your Service]"

## Positioning your skill

How you describe your skill determines whether users understand its value and actually try it. When writing about your skill—in your README, documentation, or marketing - keep these principles in mind.

### Focus on outcomes, not features:

#### ✔ Good:

"The ProjectHub skill enables teams to set up complete project workspaces in seconds — including pages, databases, and templates — instead of spending 30 minutes on manual setup."

#### ✗ Bad:

"The ProjectHub skill is a folder containing YAML frontmatter and Markdown instructions that calls our MCP server tools."

### Highlight the MCP + skills story:

"Our MCP server gives Claude access to your Linear projects. Our skills teach Claude your team's sprint planning workflow. Together, they enable AI-powered project management."

注意：API中的技能需要代码执行工具测试版，该工具提供了技能运行所需的安全环境。

有关实现细节，请参阅：

- [Skills API 快速入门](#)
- [创建自定义技能](#)
- [Agent SDK 中的技能](#)

## 当前推荐的方法

首先，将你的技能托管在 GitHub 上，拥有一个公开仓库、清晰的 README（供访客阅读——这与你的技能文件夹是分开的，技能文件夹不应包含 README.md），并提供带有截图的示例用法。然后，在你的 MCP 文档中添加一个部分，链接到该技能，解释为何两者结合使用有价值，并提供一份快速入门指南。

### 1. 托管在 GitHub

- 公开仓库用于开源技能
- 清晰的 README 包含安装说明
- 示例用法和截图

### 2. 在您的 MCP 仓库中记录

- 从 MCP 文档链接到技能
- 解释两者结合使用的价值
- 提供快速入门指南

### 3. 创建安装指南

```
## Installing the [Your Service] skill

1. Download the skill:
  - Clone repo: `git clone https://github.com/yourcompany/skills`
  - Or download ZIP from Releases

2. Install in Claude:
  - Open Claude.ai > Settings > skills
  - Click "Upload skill"
```

- Select the skill folder (zipped)

#### 3. Enable the skill:

- Toggle on the [Your Service] skill
- Ensure your MCP server is connected

#### 4. Test:

- Ask Claude: "Set up a new project in [Your Service]"

## 定位你的技能

你如何描述你的技能决定了用户是否能理解其价值并真正尝试使用。在撰写关于你的技能的内容时——无论是在README、文档还是营销材料中——请牢记以下原则。

### 聚焦于成果，而非功能：

#### ✔ 优点：

"The ProjectHub skill enables teams to set up complete project workspaces in seconds — including pages, databases, and templates — instead of spending 30 minutes on manual setup."

#### ✗ 缺点：

"The ProjectHub skill is a folder containing YAML frontmatter and Markdown instructions that calls our MCP server tools."

### 突出MCP + 技能故事：

"Our MCP server gives Claude access to your Linear projects. Our skills teach Claude your team's sprint planning workflow. Together, they enable AI-powered project management."



Chapter 5

# Patterns and troubleshooting



第5章

# 模式与故障排除

# Patterns and troubleshooting

These patterns emerged from skills created by early adopters and internal teams. They represent common approaches we've seen work well, not prescriptive templates.

## Choosing your approach: Problem-first vs. tool-first

Think of it like Home Depot. You might walk in with a problem - "I need to fix a kitchen cabinet" - and an employee points you to the right tools. Or you might pick out a new drill and ask how to use it for your specific job.

Skills work the same way:

- **Problem-first:** "I need to set up a project workspace" → Your skill orchestrates the right MCP calls in the right sequence. Users describe outcomes; the skill handles the tools.
- **Tool-first:** "I have Notion MCP connected" → Your skill teaches Claude the optimal workflows and best practices. Users have access; the skill provides expertise.

Most skills lean one direction. Knowing which framing fits your use case helps you choose the right pattern below.

## Pattern 1: Sequential workflow orchestration

**Use when:** Your users need multi-step processes in a specific order.

**Example structure:**

```
## Workflow: Onboard New Customer

### Step 1: Create Account
Call MCP tool: `create_customer`
Parameters: name, email, company

### Step 2: Setup Payment
Call MCP tool: `setup_payment_method`
Wait for: payment method verification

### Step 3: Create Subscription
Call MCP tool: `create_subscription`
Parameters: plan_id, customer_id (from Step 1)

### Step 4: Send Welcome Email
Call MCP tool: `send_email`
Template: welcome_email_template
```

**Key techniques:**

- Explicit step ordering
- Dependencies between steps
- Validation at each stage
- Rollback instructions for failures

# 模式与故障排除

这些模式源自早期采用者和内部团队创建的技能。它们代表了我们看到行之有效的常见方法，而非规定性的模板。

## 选择你的方法：问题优先 vs 工具优先

想象一下家得宝的情景。你可能带着一个问题走进商店——“我需要修理一个厨房橱柜”——然后店员会指引你找到合适的工具。或者，你可能挑选了一把新电钻，然后询问如何用它来完成你的具体工作。

技能的工作方式与此相同：

- 问题优先："我需要设置一个项目工作区" → 您的技能会按正确顺序编排正确的 MCP 调用。用户描述结果；技能处理工具。
- 工具优先："我已连接 Notion MCP" → 您的技能向 Claude 传授最优工作流和最佳实践。用户拥有访问权限；技能提供专业知识。

多数技能偏向某一方向。了解哪种框架适合您的使用场景，有助于您选择下方正确的模式。

## 模式 1：顺序工作流编排

**适用场景：** 您的用户需要按特定顺序执行多步骤流程。

**示例结构：**

```
## Workflow: Onboard New Customer

### Step 1: Create Account
Call MCP tool: `create_customer`
Parameters: name, email, company

### Step 2: Setup Payment
Call MCP tool: `setup_payment_method`
Wait for: payment method verification

### Step 3: Create Subscription
Call MCP tool: `create_subscription`
Parameters: plan_id, customer_id (from Step 1)

### Step 4: Send Welcome Email
Call MCP tool: `send_email`
Template: welcome_email_template
```

**关键技术：**

- 显式步骤排序
- 步骤间依赖
- 每个阶段的验证
- 失败回滚指令

## Pattern 2: Multi-MCP coordination

**Use when:** Workflows span multiple services.

### Example: Design-to-development handoff

```
### Phase 1: Design Export (Figma MCP)
1. Export design assets from Figma
2. Generate design specifications
3. Create asset manifest

### Phase 2: Asset Storage (Drive MCP)
1. Create project folder in Drive
2. Upload all assets
3. Generate shareable links

### Phase 3: Task Creation (Linear MCP)
1. Create development tasks
2. Attach asset links to tasks
3. Assign to engineering team

### Phase 4: Notification (Slack MCP)
1. Post handoff summary to #engineering
2. Include asset links and task references
```

#### Key techniques:

- Clear phase separation
- Data passing between MCPs
- Validation before moving to next phase
- Centralized error handling

## Pattern 3: Iterative refinement

**Use when:** Output quality improves with iteration.

### Example: Report generation

```
## Iterative Report Creation

### Initial Draft
1. Fetch data via MCP
2. Generate first draft report
3. Save to temporary file

### Quality Check
1. Run validation script: `scripts/check_report.py`
2. Identify issues:
   - Missing sections
   - Inconsistent formatting
   - Data validation errors

### Refinement Loop
1. Address each identified issue
2. Regenerate affected sections
3. Re-validate
4. Repeat until quality threshold met

### Finalization
1. Apply final formatting
2. Generate summary
3. Save final version
```

#### Key techniques:

- Explicit quality criteria
- Iterative improvement
- Validation scripts
- Know when to stop iterating

## 模式2：多MCP协调

适用场景：工作流跨越多个服务。

### 示例：设计到开发交接

```
### Phase 1: Design Export (Figma MCP)
1. Export design assets from Figma
2. Generate design specifications
3. Create asset manifest

### Phase 2: Asset Storage (Drive MCP)
1. Create project folder in Drive
2. Upload all assets
3. Generate shareable links

### Phase 3: Task Creation (Linear MCP)
1. Create development tasks
2. Attach asset links to tasks
3. Assign to engineering team

### Phase 4: Notification (Slack MCP)
1. Post handoff summary to #engineering
2. Include asset links and task references
```

#### 关键技术：

- 清晰的阶段分离
- MCP间的数据传递
- 进入下一阶段前的验证
- 集中式错误处理

## 模式 3：迭代优化

适用场景：输出质量通过迭代得以提升。

### 示例：报告生成

```
## Iterative Report Creation

### Initial Draft
1. Fetch data via MCP
2. Generate first draft report
3. Save to temporary file

### Quality Check
1. Run validation script: `scripts/check_report.py`
2. Identify issues:
   - Missing sections
   - Inconsistent formatting
   - Data validation errors

### Refinement Loop
1. Address each identified issue
2. Regenerate affected sections
3. Re-validate
4. Repeat until quality threshold met

### Finalization
1. Apply final formatting
2. Generate summary
3. Save final version
```

#### 关键技术：

- 明确的质量标准
- 迭代改进
- 验证脚本
- 知道何时停止迭代

## Pattern 4: Context-aware tool selection

**Use when:** Same outcome, different tools depending on context.

**Example:** File storage

```
## Smart File Storage

### Decision Tree
1. Check file type and size
2. Determine best storage location:
  - Large files (>10MB): Use cloud storage MCP
  - Collaborative docs: Use Notion/Docs MCP
  - Code files: Use GitHub MCP
  - Temporary files: Use local storage

### Execute Storage
Based on decision:
- Call appropriate MCP tool
- Apply service-specific metadata
- Generate access link

### Provide Context to User
Explain why that storage was chosen
```

**Key techniques:**

- Clear decision criteria
- Fallback options
- Transparency about choices

## Pattern 5: Domain-specific intelligence

**Use when:** Your skill adds specialized knowledge beyond tool access.

**Example:** Financial compliance

```
## Payment Processing with Compliance

### Before Processing (Compliance Check)
1. Fetch transaction details via MCP
2. Apply compliance rules:
  - Check sanctions lists
  - Verify jurisdiction allowances
  - Assess risk level
3. Document compliance decision

### Processing
IF compliance passed:
  - Call payment processing MCP tool
  - Apply appropriate fraud checks
  - Process transaction
ELSE:
  - Flag for review
  - Create compliance case

### Audit Trail
- Log all compliance checks
- Record processing decisions
- Generate audit report
```

**Key techniques:**

- Domain expertise embedded in logic
- Compliance before action
- Comprehensive documentation
- Clear governance

## 模式 4：上下文感知工具选择

使用场景：目标相同，但根据上下文选择不同的工具。

示例：文件存储

```
## Smart File Storage

### Decision Tree
1. Check file type and size
2. Determine best storage location:
  - Large files (>10MB): Use cloud storage MCP
  - Collaborative docs: Use Notion/Docs MCP
  - Code files: Use GitHub MCP
  - Temporary files: Use local storage

### Execute Storage
Based on decision:
- Call appropriate MCP tool
- Apply service-specific metadata
- Generate access link

### Provide Context to User
Explain why that storage was chosen
```

关键技术：

- 清晰的决策标准
- 备用选项
- 关于选择的透明度

## 模式 5：领域特定智能

使用场景：当你的技能提供了超越工具访问的专门知识时。

示例：金融合规

```
## Payment Processing with Compliance

### Before Processing (Compliance Check)
1. Fetch transaction details via MCP
2. Apply compliance rules:
  - Check sanctions lists
  - Verify jurisdiction allowances
  - Assess risk level
3. Document compliance decision

### Processing
IF compliance passed:
  - Call payment processing MCP tool
  - Apply appropriate fraud checks
  - Process transaction
ELSE:
  - Flag for review
  - Create compliance case

### Audit Trail
- Log all compliance checks
- Record processing decisions
- Generate audit report
```

关键技术：

- 嵌入逻辑中的领域专业知识
- 行动前合规
- 全面的文档
- 清晰的治理

## Troubleshooting

### Skill won't upload

**Error: "Could not find SKILL.md in uploaded folder"**

Cause: File not named exactly SKILL.md

**Solution:**

- Rename to SKILL.md (case-sensitive)
- Verify with: `ls -la` should show SKILL.md

**Error: "Invalid frontmatter"**

Cause: YAML formatting issue

Common mistakes:

```
# Wrong - missing delimiters
name: my-skill
description: Does things

# Wrong - unclosed quotes
name: my-skill
description: "Does things

# Correct
---
name: my-skill
description: Does things
---
```

**Error: "Invalid skill name"**

Cause: Name has spaces or capitals

```
# Wrong
name: My Cool Skill

# Correct
name: my-cool-skill
```

### Skill doesn't trigger

**Symptom:** Skill never loads automatically

**Fix:**

Revise your description field. See The Description Field for good/bad examples.

**Quick checklist:**

- Is it too generic? ("Helps with projects" won't work)
- Does it include trigger phrases users would actually say?
- Does it mention relevant file types if applicable?

**Debugging approach:**

Ask Claude: "When would you use the [skill name] skill?" Claude will quote the description back. Adjust based on what's missing.

### Skill triggers too often

**Symptom:** Skill loads for unrelated queries

**Solutions:**

1. **Add negative triggers**

```
description: Advanced data analysis for CSV files. Use for
statistical modeling, regression, clustering. Do NOT use for
simple data exploration (use data-viz skill instead).
```

## 故障排除

### 技能无法上传

错误: "无法在上传的文件夹中找到 SKILL.md"

原因: 文件未精确命名为 SKILL.md

解决方案:

- 重命名为 SKILL.md (大小写敏感)
- 验证: `ls -la` 应显示 SKILL.md

错误: “无效的前置元数据”

原因: YAML 格式问题

常见错误:

```
# Wrong - missing delimiters
name: my-skill
description: Does things

# Wrong - unclosed quotes
name: my-skill
description: "Does things

# Correct
---
name: my-skill
description: Does things
---
```

错误: “无效的技能名称”

原因: 姓名包含空格或大写字母

```
# Wrong
name: My Cool Skill

# Correct
name: my-cool-skill
```

### 技能未触发

症状: 技能从未自动加载

**Fix:**

修订您的描述字段。请参阅《描述字段》了解好坏示例。

快速检查清单:

- 是否过于笼统? (“协助项目”无效)
- 是否包含用户实际会说的触发短语?
- 是否提及了相关的文件类型 (如适用)?

调试方法:

询问Claude: “何时会使用[技能名称] 技能?” Claude会引用description回复。根据缺失内容进行调整。

### 技能触发过于频繁

症状: 技能因无关查询而加载

解决方案:

1. 添加否定触发词

```
description: Advanced data analysis for CSV files. Use for
statistical modeling, regression, clustering. Do NOT use for
simple data exploration (use data-viz skill instead).
```

2. Be more specific

```
# Too broad
description: Processes documents

# More specific
description: Processes PDF legal documents for contract review
```

3. Clarify scope

```
description: PayFlow payment processing for e-commerce. Use
specifically for online payment workflows, not for general
financial queries.
```

MCP connection issues

**Symptom:** Skill loads but MCP calls fail

Checklist:

- Verify MCP server is connected**
  - Claude.ai: Settings > Extensions > [Your Service]
  - Should show "Connected" status
- Check authentication**
  - API keys valid and not expired
  - Proper permissions/scopes granted
  - OAuth tokens refreshed
- Test MCP independently**
  - Ask Claude to call MCP directly (without skill)
  - "Use [Service] MCP to fetch my projects"
  - If this fails, issue is MCP not skill
- Verify tool names**
  - Skill references correct MCP tool names
  - Check MCP server documentation
  - Tool names are case-sensitive

Instructions not followed

**Symptom:** Skill loads but Claude doesn't follow instructions

Common causes:

- Instructions too verbose**
  - Keep instructions concise
  - Use bullet points and numbered lists
  - Move detailed reference to separate files
- Instructions buried**
  - Put critical instructions at the top
  - Use ## Important or ## Critical headers
  - Repeat key points if needed

3. Ambiguous language

```
# Bad
Make sure to validate things properly

# Good
CRITICAL: Before calling create_project, verify:
- Project name is non-empty
- At least one team member assigned
- Start date is not in the past
```

**Advanced technique:** For critical validations, consider bundling a script that performs the checks programmatically rather than relying on language instructions. Code is deterministic; language interpretation isn't. See the [Office skills](#) for examples of this pattern.

4. Model "laziness" Add explicit encouragement:

```
## Performance Notes
- Take your time to do this thoroughly
- Quality is more important than speed
- Do not skip validation steps
```

Note: Adding this to user prompts is more effective than in SKILL.md

2. 更具体一些

```
# Too broad
description: Processes documents

# More specific
description: Processes PDF legal documents for contract review
```

3. 明确范围

```
description: PayFlow payment processing for e-commerce. Use
specifically for online payment workflows, not for general
financial queries.
```

MCP 连接问题

症状：技能已加载但 MCP 调用失败

检查清单：

- 验证 MCP 服务器已连接
  - Claude.ai：设置 > 扩展 > [您的服务]
  - OAuth令牌已刷新
- 检查认证
  - API密钥有效且未过期
  - 已授予适当权限/作用域
  - OAuth tokens refreshed
- 独立测试 MCP
  - 检查 MCP 服务器文档
  - "使用[服务] MCP来获取我的项目"
  - 如果失败，问题在于 MCP 而非技能
- 验证工具名称
  - 技能引用了正确的 MCP 工具名称
  - Check MCP server documentation
  - 工具名称区分大小写

未遵循指令

症状：技能已加载但Claude未遵循指令

常见原因：

- 指令过于冗长
  - 保持指令简洁
  - 使用要点和编号列表
  - 将详细参考资料移至单独文件
- 指令被埋藏
  - 将关键指令置于顶部
  - 使用 ## 重要或 ## 关键标题
  - 必要时重复关键点

3. 语言表述模糊

```
# Bad
Make sure to validate things properly

# Good
CRITICAL: Before calling create_project, verify:
- Project name is non-empty
- At least one team member assigned
- Start date is not in the past
```

高级技巧：对于关键验证，可以考虑捆绑一个脚本，以程序化方式执行检查，而非依赖语言指令。代码是确定性的；语言解释则不然。请参阅 Office 技能中的示例以了解此模式。

4. 模型“惰性”添加明确鼓励：

```
## Performance Notes
- Take your time to do this thoroughly
- Quality is more important than speed
- Do not skip validation steps
```

注意：将此添加到用户提示中比在SKILL.md中更有效

## Large context issues

**Symptom:** Skill seems slow or responses degraded

**Causes:**

- Skill content too large
- Too many skills enabled simultaneously
- All content loaded instead of progressive disclosure

**Solutions:**

1. **Optimize SKILL.md size**
  - Move detailed docs to references/
  - Link to references instead of inline
  - Keep SKILL.md under 5,000 words
2. **Reduce enabled skills**
  - Evaluate if you have more than 20 - 50 skills enabled simultaneously
  - Recommend selective enablement
  - Consider skill "packs" for related capabilities

## 大型上下文问题

症状：技能似乎运行缓慢或响应质量下降

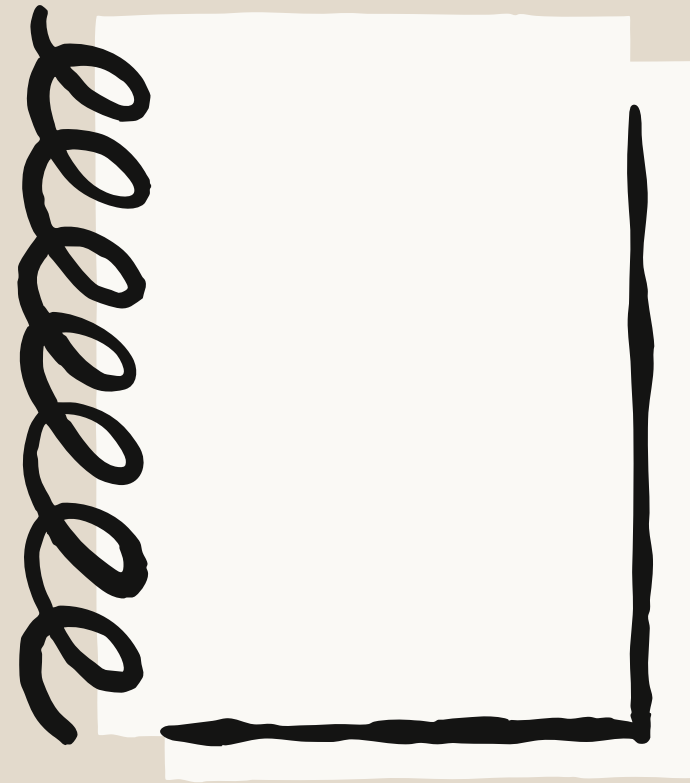
原因：

- 技能内容过大
- 同时启用过多技能
- 加载全部内容而非采用渐进式披露

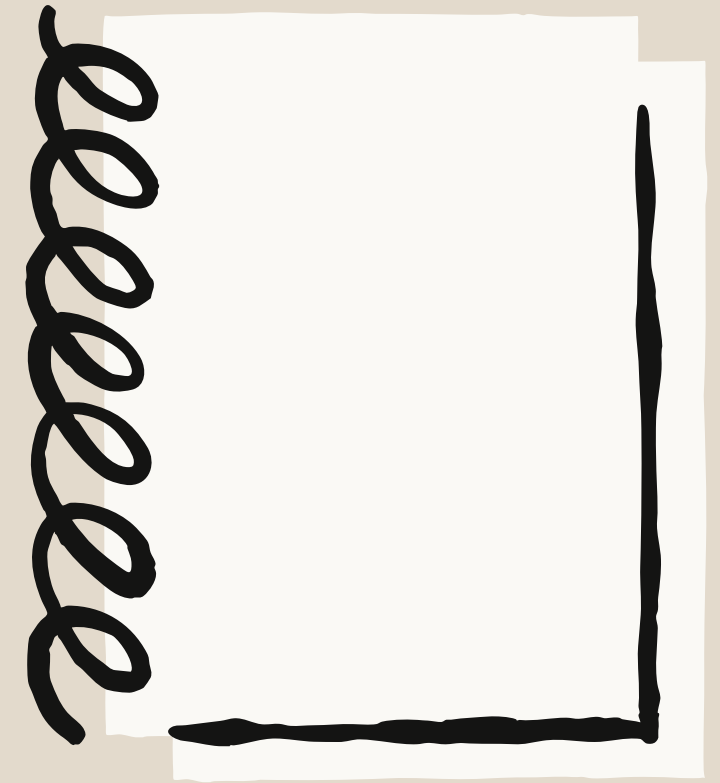
解决方案：

1. 优化 SKILL.md 文件大小
  - 将详细文档移至 references/ 目录
  - 链接至参考资料而非内嵌
  - 保持 SKILL.md 字数在 5,000 以内
2. 减少启用技能数量
  - 评估是否同时启用了超过 20 - 50 个技能
  - 建议选择性启用
  - 考虑为相关Capabilities提供技能"包"

# Resources and references



# 资源与 references



# Resources and references

If you're building your first skill, start with the Best Practices Guide, then reference the API docs as needed.

## Official Documentation

### Anthropic Resources:

- [Best Practices Guide](#)
- [Skills Documentation](#)
- [API Reference](#)
- [MCP Documentation](#)

### Blog Posts:

- [Introducing Agent Skills](#)
- [Engineering Blog: Equipping Agents for the Real World](#)
- [Skills Explained](#)
- [How to Create Skills for Claude](#)
- [Building Skills for Claude Code](#)
- [Improving Frontend Design through Skills](#)

## Example skills

### Public skills repository:

- GitHub: [anthropics/skills](#)
- Contains Anthropic-created skills you can customize

## Tools and Utilities

### skill-creator skill:

- Built into Claude.ai and available for Claude Code
- Can generate skills from descriptions
- Reviews and provides recommendations
- Use: "Help me build a skill using skill-creator"

### Validation:

- skill-creator can assess your skills
- Ask: "Review this skill and suggest improvements"

## Getting Support

### For Technical Questions:

- General questions: Community forums at the [Claude Developers Discord](#)

### For Bug Reports:

- GitHub Issues: [anthropics/skills/issues](#)
- Include: Skill name, error message, steps to reproduce

# 资源与references

如果你正在构建你的第一个技能，请从最佳实践指南开始，然后根据需要参考API文档。

## 官方文档

### Anthropic资源：

- [最佳实践指南](#)
- [技能文档](#)
- [API参考](#)
- [MCP 文档](#)

### 博客文章：

- [智能体技能介绍](#)
- [工程博客：为智能体配备现实世界能力](#)
- [技能详解](#)
- [如何为 Claude 创建技能](#)
- [为 Claude Code 构建技能](#)
- [通过技能改进前端设计](#)

## 示例技能

### 公共技能仓库：

- GitHub: [anthropics/skills](#)
- 包含可供您自定义的 Anthropic 创建的技能

## 工具与实用程序

### 技能创建器技能：

- 内置于 Claude.ai 中，并可供 Claude Code 使用
- 可根据描述生成技能
- 审查并提供建议
- 用法示例: "帮我使用 skill-creator 构建一个技能"

### 验证：

- skill-creator 可以评估您的技能
- 询问示例: "审查此技能并提出改进建议"

## 获取支持

### 技术问题咨询：

- 一般性问题: 请前往 Claude 开发者 Discord [社区论坛](#)

### Bug 报告：

- GitHub Issues: [anthropics/skills/issues](#)
- 请包含: 技能名称、错误信息、复现步骤

# Reference A: Quick checklist

Use this checklist to validate your skill before and after upload. If you want a faster start, use the skill-creator skill to generate your first draft, then run through this list to make sure you haven't missed anything.

## Before you start

- ☐ Identified 2-3 concrete use cases
- ☐ Tools identified (built-in or MCP)
- ☐ Reviewed this guide and example skills
- ☐ Planned folder structure

## During development

- ☐ Folder named in kebab-case
- ☐ SKILL.md file exists (exact spelling)
- ☐ YAML frontmatter has --- delimiters
- ☐ name field: kebab-case, no spaces, no capitals
- ☐ description includes WHAT and WHEN
- ☐ No XML tags (<>) anywhere
- ☐ Instructions are clear and actionable
- ☐ Error handling included
- ☐ Examples provided
- ☐ References clearly linked

## Before upload

- ☐ Tested triggering on obvious tasks
- ☐ Tested triggering on paraphrased requests
- ☐ Verified doesn't trigger on unrelated topics
- ☐ Functional tests pass
- ☐ Tool integration works (if applicable)
- ☐ Compressed as .zip file

## After upload

- ☐ Test in real conversations
- ☐ Monitor for under/over-triggering
- ☐ Collect user feedback
- ☐ Iterate on description and instructions
- ☐ Update version in metadata

# 参考A：快速checlist

使用此清单在上传前后验证您的技能。如果您希望更快上手，可使用技能创建器技能生成您的初稿，然后运行此清单以确保您没有遗漏任何内容。

## 开始之前

- ☐ 已确定 2-3 个具体用例
- ☐ 已确定工具（内置或 MCP）
- ☐ 已审阅本指南及示例技能
- ☐ 已规划文件夹结构

## 开发期间

- ☐ 文件夹名称采用短横线命名法
- ☐ SKILL.md 文件存在（拼写准确）
- ☐ YAML 前置元数据有 --- 分隔符
- ☐ name 字段：短横线命名法，无空格，无大写字母
- ☐ description 包含 WHAT 和 WHEN
- ☐ 任何地方都没有 XML 标签 (<>)
- ☐ 指令清晰且可操作
- ☐ 包含错误处理
- ☐ 提供示例
- ☐ References 清晰链接

## 上传前

- ☐ 测试在明显任务上的触发
- ☐ 测试在改写请求上的触发
- ☐ Verified 不会在无关主题上触发
- ☐ 功能测试通过
- ☐ 工具集成正常工作（如适用）
- ☐ 压缩为 .zip 文件

## 上传后

- ☐ 在实际对话中进行测试
- ☐ 监控触发不足/过度触发的情况
- ☐ 收集用户反馈
- ☐ 迭代优化描述和说明
- ☐ 更新元数据中的版本

# Reference B: YAML frontmatter

## Required fields

```
---
name: skill-name-in-kebab-case
description: What it does and when to use it. Include specific trigger phrases.
---
```

## All optional fields

```
name: skill-name
description: [required description]
license: MIT # Optional: License for open-source
allowed-tools: "Bash(python:*) Bash(npm:*) WebFetch" # Optional: Restrict tool access
metadata: # Optional: Custom fields
  author: Company Name
  version: 1.0.0
  mcp-server: server-name
  category: productivity
  tags: [project-management, automation]
documentation: https://example.com/docs
support: support@example.com
```

## Security notes

### Allowed:

- Any standard YAML types (strings, numbers, booleans, lists, objects)
- Custom metadata fields
- Long descriptions (up to 1024 characters)

### Forbidden:

- XML angle brackets (< >) - security restriction
- Code execution in YAML (uses safe YAML parsing)
- Skills named with "claude" or "anthropic" prefix (reserved)

# 参考 B：YAML 前置元数据

## 必填字段

```
---
name: skill-name-in-kebab-case
description: What it does and when to use it. Include specific trigger phrases.
---
```

## 所有可选字段

```
name: skill-name
description: [required description]
license: MIT # Optional: License for open-source
allowed-tools: "Bash(python:*) Bash(npm:*) WebFetch" # Optional: Restrict tool access
metadata: # Optional: Custom fields
  author: Company Name
  version: 1.0.0
  mcp-server: server-name
  category: productivity
  tags: [project-management, automation]
documentation: https://example.com/docs
support: support@example.com
```

## 安全说明

### 允许:

- 任何标准 YAML 类型（字符串、数字、布尔值、列表、对象）
- 自定义元数据字段
- 长描述（最多1024个字符）

### 禁止项:

- XML 尖括号 (< >) - 安全限制
- YAML 中的代码执行（使用安全 YAML 解析）
- 以 "claude" 或 "anthropic" 前缀命名的技能（保留项）

# Reference C: Complete skill examples

For full, production-ready skills demonstrating the patterns in this guide:

- Document Skills - [PDF](#), [DOCX](#), [PPTX](#), [XLSX](#) creation
- [Example Skills](#) - Various workflow patterns
- [Partner Skills Directory](#) - View skills from various partners such as Asana, Atlassian, Canva, Figma, Sentry, Zapier, and more

These repositories stay up-to-date and include additional examples beyond what's covered here. Clone them, modify them for your use case, and use them as templates.

# 参考C：完整的技能示例

如需完整的、可用于生产环境的技能，以演示本指南中的模式：

- 文档技能 - [PDF、DOCX、PPTX、XLSX创建](#)
- [示例技能](#) - 各种工作流模式
- [合作伙伴技能目录](#) - [查看来自Asana等各类合作伙伴的技能](#)  
Atlassian、Canva、Figma、Sentry、Zapier等

这些仓库始终保持更新，并包含超出此处所涵盖内容的额外示例。克隆它们，根据您的用例进行修改，并将其用作模板。



claude.ai



Claude.ai